Microservices Learning Guide

Table of Contents

Redis

Theory
Redis is an in-memory data structure store used as a database, cache, and message broker. It supports various data structures like strings, hashes, lists, sets, and sorted sets.

Key Features
- **In-memory storage** - Extremely fast operations
- **Data persistence** - Optional disk persistence
- **Data structures** - Strings, Hashes, Lists, Sets, Sorted Sets
- **Pub/Sub** - Message publishing/subscription
- **Transactions** - Atomic operations
- **Lua scripting** - Server-side scripting

Use Cases
- **Caching** - Frequently accessed data
- **Session storage** - User sessions
- **Leaderboards** - Sorted sets for rankings
- **Real-time analytics** - Counters and metrics

- **Message queue** - Simple pub/sub

Implementation in Your Project

Configuration (appsettings.json)
```json
{
  "CacheSettings": {
    "ConnectionString": "basketdb:6379"
  }
}
```

Program.cs Configuration
```csharp
// Program.cs (lines 87-92)
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration =
builder.Configuration["CacheSettings:ConnectionString"];
});

builder.Services.AddScoped<IBasketRepository, BasketRepository>();
```

Repository Implementation
```csharp
// Basket.Infrastructure/Repositories/BasketRepository.cs
public class BasketRepository : IBasketRepository
{
    private readonly IDistributedCache _redisCache;

    public async Task<ShoppingCart> GetBasket(string userName)
    {
        var basket = await _redisCache.GetStringAsync(userName);
        if(string.IsNullOrEmpty(basket))
        {
            return null;
        }
        return JsonConvert.DeserializeObject<ShoppingCart>(basket);
    }

    public async Task<ShoppingCart> UpdateBasket(ShoppingCart shoppingCart)
```

```
    {
        await _redisCache.SetStringAsync(shoppingCart.UserName,
JsonConvert.SerializeObject(shoppingCart));
        return await GetBasket(shoppingCart.UserName);
    }

    public async Task DeleteBasket(string userName)
    {
        await _redisCache.RemoveAsync(userName);
    }
}
```

Docker Configuration
```yaml
# docker-compose.override.yml
basketdb:
  container_name: basketdb
  restart: always
  ports:
    - "6379:6379"
```

Benefits in Your Project
- **Fast basket retrieval** - In-memory storage for shopping carts
- **Session persistence** - User baskets stored in Redis
- **Scalability** - Redis cluster support for high availability
- **Distributed caching** - Multiple API instances share cache

---

gRPC

Theory
gRPC (Google Remote Procedure Call) is a high-performance, open-source RPC framework that uses HTTP/2 for transport, Protocol Buffers for serialization, and supports multiple programming languages.

Key Features
- **HTTP/2** - Multiplexed connections, header compression
- **Protocol Buffers** - Binary serialization, smaller payloads
- **Code generation** - Auto-generated client/server code
- **Streaming** - Client, server, and bidirectional streaming

```
- **Performance** - 5-10x faster than REST/JSON

### Protocol Buffers (.proto)
```protobuf
syntax = "proto3";

option csharp_namespace = "Discount.Grpc.Protos";

service DiscountProtoService {
  rpc GetDiscount(GetDiscountRequest) returns (CouponModel);
  rpc CreateDiscount(CreateDiscountRequest) returns (CouponModel);
  rpc UpdateDiscount(UpdateDiscountRequest) returns (CouponModel);
  rpc DeleteDiscount(DeleteDiscountRequest) returns (DeleteCouponResponse);
}

message GetDiscountRequest {
  string productName = 1;
}

message CouponModel {
  int32 id = 1;
  string productName = 2;
  string description = 3;
  int32 amount = 4;
}
```

Implementation in Your Project

Discount Service (Server)
```csharp
// Discount.API/Services/DiscountService.cs
public class DiscountService : DiscountProtoService.DiscountProtoServiceBase
{
    private readonly IMediator _mediator;

    public override async Task<CouponModel> GetDiscount(GetDiscountRequest
request, ServerCallContext context)
    {
        var query = new GetDiscountQuery(request.ProductName);
        var result = await _mediator.Send(query);
        return result;
    }
```
```

```
}
```

Program.cs Configuration
```csharp
// Discount.API/Program.cs (lines 49-57)
builder.Services.AddGrpc();
builder.Services.AddScoped<IDiscountRepository, DiscountRepository>();
builder.Services.AddMediatR(typeof(Program));

app.MapGrpcService<DiscountService>();
```

Basket Service (Client)
```csharp
// Basket.API/Program.cs (lines 62-69)
builder.Services.AddGrpcClient<DiscountProtoService.DiscountProtoServiceClient>(o
=>
{
    o.Address = new Uri(builder.Configuration["GrpcSettings:DiscountUrl"]);
});
```

gRPC Service Client
```csharp
// Basket.Application/GrpcService/DiscountGrpcService.cs
public class DiscountGrpcService
{
    private readonly DiscountProtoService.DiscountProtoServiceClient
_discountProtoServiceClient;

    public async Task<CouponModel> GetDiscount(string productName)
    {
        var discountRequest = new GetDiscountRequest { ProductName =
productName };
        return await
_discountProtoServiceClient.GetDiscountAsync(discountRequest);
    }
}
```

Usage in Command Handler
```csharp
```

```
// Basket.Application/Handlers/CreateShoppingCartCommandHandler.cs
public async Task<ShoppingCartResponse> Handle(CreateShoppingCartCommand request,
CancellationToken cancellationToken)
{
    foreach (var item in request.Items)
    {
        var coupon = await _discountGrpcService.GetDiscount(item.ProductName);
        item.Price -= coupon.Amount;
    }
    // ... rest of implementation
}
```

Benefits in Your Project
- **High performance** - Binary serialization, HTTP/2
- **Type safety** - Compile-time checking
- **Code generation** - Auto-generated client/server stubs
- **Streaming** - Support for bidirectional streaming
- **Cross-language** - Multiple programming language support

HTTP REST API vs gRPC Communication

Comparison

Aspect	HTTP REST API	gRPC
Protocol	HTTP/1.1	HTTP/2
Serialization	JSON/XML	Protocol Buffers
Performance	Good	Excellent
Code Generation	None	Auto-generated
Streaming	Limited	Full support
Browser Support	Excellent	Limited
Tooling	Mature	Growing

When to Use REST API
- **Frontend communication** - Browser compatibility
- **Public APIs** - External consumers
- **Simple CRUD** - Basic operations
- **Tooling** - Postman, Swagger, curl

When to Use gRPC

- **Inter-service communication** - Microservices
- **High performance** - Low latency required
- **Streaming** - Real-time data
- **Type safety** - Compile-time contracts

Hybrid Approach (Your Project)
```
Frontend (Angular) → REST API → Basket Service → gRPC → Discount Service
```

Benefits:
- **Frontend** - Uses familiar REST APIs
- **Inter-service** - Uses high-performance gRPC
- **Best of both worlds** - Browser compatibility + performance

---

Background Services (Hangfire)

Theory
Hangfire is an open-source framework that helps you create, process, and manage background jobs in .NET applications. It provides a simple way to run background tasks without worrying about threading, reliability, and persistence.

Key Features
- **Persistent storage** - Jobs survive application restarts
- **Automatic retries** - Configurable retry policies
- **Dashboard** - Web UI for monitoring
- **Fire-and-forget** - One-time jobs
- **Recurring jobs** - Scheduled tasks
- **Continuations** - Job chaining

Job Types

Fire-and-Forget
```csharp
// Execute once and forget
BackgroundJob.Enqueue(() => SendEmail(user.Email, "Welcome!"));
```

Delayed
```csharp
```

```
// Execute after delay
BackgroundJob.Schedule(() => SendReminder(user.Id), TimeSpan.FromDays(1));
```

Recurring
```csharp
// Execute on schedule
RecurringJob.AddOrUpdate("daily-report", () => GenerateDailyReport(),
Cron.Daily);
```

Continuations
```csharp
// Execute after parent job
var jobId = BackgroundJob.Enqueue(() => ProcessData(data));
BackgroundJob.ContinueJobWith(jobId, () => SendNotification(data.Id));
```

Implementation (Not in Your Project)

Installation
```bash
dotnet add package Hangfire
dotnet add package Hangfire.AspNetCore
dotnet add package Hangfire.SqlServer
```

Configuration
```csharp
// Program.cs
builder.Services.AddHangfire(config =>
{
    config.UseSimpleAssemblyNameTypeSerializer();
    config.UseRecommendedSerializerSettings();

config.UseSqlServerStorage(builder.Configuration.GetConnectionString("DefaultConn
ection"));
});

builder.Services.AddHangfireServer();
app.UseHangfireDashboard();
```

Background Service Class

```csharp
public class EmailService
{
    public void SendEmail(string to, string subject)
    {
        // Send email logic
        Console.WriteLine($"Email sent to {to} with subject {subject}");
    }
}

public class ReportService
{
    public void GenerateDailyReport()
    {
        // Generate report logic
        Console.WriteLine("Daily report generated");
    }
}
```

Usage in Controllers

```csharp
[ApiController]
public class OrderController : ControllerBase
{
    private readonly IBackgroundJobClient _backgroundJobClient;

    public async Task<IActionResult> CreateOrder(Order order)
    {
        // Create order
        await _orderService.CreateOrder(order);

        // Send confirmation email in background
        _backgroundJobClient.Enqueue<IEmailService>(x =>
x.SendOrderConfirmation(order.Id));

        return Ok(order);
    }
}
```

Benefits for Microservices

- **Decoupling** - Background processing doesn't block main flow
- **Reliability** - Jobs persist and retry automatically
- **Monitoring** - Dashboard for job status
- **Scalability** - Multiple workers process jobs
- **Scheduling** - Recurring tasks for maintenance

Polly & Resilience Patterns

Theory
Polly is a .NET resilience and transient-fault-handling library that allows developers to express policies such as Retry, Circuit Breaker, Timeout, Bulkhead Isolation, and Fallback in a fluent and thread-safe manner.

Resilience Patterns

1. Retry Pattern
Retries an operation a specified number of times with delays.

```csharp
var retryPolicy = Policy
    .Handle<HttpRequestException>()
    .OrResult<HttpResponseMessage>(r => !r.IsSuccessStatusCode)
    .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)));

var response = await retryPolicy.ExecuteAsync(() => httpClient.GetAsync(url));
```

2. Circuit Breaker Pattern
Stops making calls to a failing service after a threshold.

```csharp
var circuitBreakerPolicy = Policy
    .Handle<HttpRequestException>()
    .CircuitBreakerAsync(
        exceptionsAllowedBeforeBreaking: 3,
        durationOfBreak: TimeSpan.FromSeconds(30)
    );
```

```
var response = await circuitBreakerPolicy.ExecuteAsync(() =>
httpClient.GetAsync(url));
```

3. Timeout Pattern
Fails fast if operation takes too long.

```csharp
var timeoutPolicy = Policy.TimeoutAsync(TimeSpan.FromSeconds(10));

var response = await timeoutPolicy.ExecuteAsync(() => httpClient.GetAsync(url));
```

4. Fallback Pattern
Provides alternative response when primary fails.

```csharp
var fallbackPolicy = Policy<HttpResponseMessage>
    .Handle<Exception>()
    .FallbackAsync(new HttpResponseMessage(HttpStatusCode.OK)
    {
        Content = new StringContent("{\"error\":\"Service unavailable\"}",
Encoding.UTF8, "application/json")
    });

var response = await fallbackPolicy.ExecuteAsync(() => httpClient.GetAsync(url));
```

5. Bulkhead Pattern
Limits concurrent calls to prevent resource exhaustion.

```csharp
var bulkheadPolicy = Policy.BulkheadAsync(10, 100); // 10 concurrent, 100 queued

var response = await bulkheadPolicy.ExecuteAsync(() => httpClient.GetAsync(url));
```

Policy Wrapping
Combine multiple policies:

```csharp
var policyWrap = Policy.WrapAsync(
```

```
    fallbackPolicy,
    circuitBreakerPolicy,
    retryPolicy,
    timeoutPolicy
);

var response = await policyWrap.ExecuteAsync(() => httpClient.GetAsync(url));
```

Implementation in Your Project

Installation
```bash
dotnet add package Polly
```

Configuration (Not Currently Implemented)
```csharp
// Program.cs
builder.Services.AddHttpClient<IDiscountService, DiscountService>(client =>
{
    client.BaseAddress = new Uri("https://discount.api");
})
.AddTransientHttpErrorPolicy(p => p.WaitAndRetryAsync(3, _ =>
TimeSpan.FromSeconds(1)))
.AddTransientHttpErrorPolicy(p => p.CircuitBreakerAsync(3,
TimeSpan.FromSeconds(30)));
```

gRPC Resilience
```csharp
// Program.cs
builder.Services.AddGrpcClient<DiscountProtoService.DiscountProtoServiceClient>(o
=>
{
    o.Address = new Uri(builder.Configuration["GrpcSettings:DiscountUrl"]);
})
.ConfigureChannel(o =>
{
    o.HttpHandler = new PollyHandler(new ResiliencePipeline<HttpResponseMessage>
    {
        // Add resilience policies
    });
```

```
});
```

Benefits for Microservices
- **Fault tolerance** - Handles transient failures
- **Prevents cascading failures** - Circuit breaker stops propagation
- **Improves user experience** - Retries and fallbacks
- **Resource protection** - Bulkhead prevents overload
- **Observability** - Policy execution metrics

Elasticsearch

Theory
Elasticsearch is a distributed, RESTful search and analytics engine capable of addressing a growing number of use cases. It provides a distributed, multitenant-capable full-text search engine with an HTTP web interface and schema-free JSON documents.

Key Features
- **Full-text search** - Powerful text search capabilities
- **Near real-time (NRT)** - Data searchable within 1 second
- **Distributed** - Scalable across multiple nodes
- **RESTful API** - JSON over HTTP
- **Schema-free** - Dynamic mapping
- **Aggregations** - Data analytics and aggregation

Use Cases
- **Log aggregation** - Centralized logging
- **Full-text search** - Product search, document search
- **Real-time analytics** - Metrics, dashboards
- **Security analytics** - SIEM, threat detection
- **Application performance monitoring** - APM

Implementation in Your Project

Configuration (appsettings.json)
```json
{
  "ElasticConfiguration": {
    "Uri": "http://elasticsearch:9200"
  }
```

```
}
```

Serilog Configuration
```csharp
// Infrastructure/Common.Logging/Logging.cs
public static void ConfigureLogging(WebApplicationBuilder builder, string
serviceName)
{
    builder.Host.UseSerilog((context, loggerConfiguration) =>
    {
        loggerConfiguration.WriteTo.Console();

        var elasticUrl =
context.Configuration.GetValue<string>("ElasticConfiguration:Uri");
        if (!string.IsNullOrEmpty(elasticUrl))
        {
            loggerConfiguration.WriteTo.Elasticsearch(
                new ElasticsearchSinkOptions(new Uri(elasticUrl))
                {
                    AutoRegisterTemplate = true,
                    AutoRegisterTemplateVersion =
AutoRegisterTemplateVersion.ESv8,
                    IndexFormat = $"{serviceName}-Logs-{0:yyyy.MM.dd}",
                    MinimumLogEventLevel = LogEventLevel.Debug
                });
        }
    });
}
```

Docker Configuration
```yaml
# docker-compose.override.yml
elasticsearch:
  container_name: elasticsearch
  environment:
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
    - discovery.type=single-node
    - xpack.security.enabled=false
  ports:
    - "9200:9200"
  volumes:
```

```
    - elasticsearch-data:/usr/share/elasticsearch/data

kibana:
  container_name: kibana
  environment:
    - ELASTICSEARCH_URL=http://elasticsearch:9200
  depends_on:
    - elasticsearch
  ports:
    - "5601:5601"
```

Usage in Services
```csharp
// Program.cs (all services)
Logging.ConfigureLogging(builder, "Basket.API");
```

Kibana Access
```
http://localhost:5601
```

Benefits in Your Project
- **Centralized logging** - All services log to Elasticsearch
- **Log aggregation** - Search and filter logs across services
- **Real-time monitoring** - View logs as they arrive
- **Kibana dashboard** - Visualize logs and metrics
- **Distributed tracing** - Correlate logs across services

RabbitMQ

Theory
RabbitMQ is a message broker that implements the Advanced Message Queuing Protocol (AMQP). It acts as a middleman for communication between applications, providing reliable message delivery, routing, and queuing.

Key Concepts
- **Producer** - Application that sends messages
- **Consumer** - Application that receives messages
- **Queue** - Buffer that stores messages

- **Exchange** - Routes messages to queues
- **Binding** - Link between exchange and queue
- **Routing Key** - Message routing information

Message Patterns

Direct Exchange
Routes messages to queues with exact routing key match.

Topic Exchange
Routes messages to queues with wildcard pattern matching.

Fanout Exchange
Routes messages to all bound queues (broadcast).

Headers Exchange
Routes messages based on message headers.

Implementation in Your Project

Configuration (appsettings.json)
```json
{
  "EventBusSettings": {
    "HostAddress": "amqp://guest:guest@rabbitmq:5672"
  }
}
```

MassTransit Configuration
```csharp
// Program.cs (Basket.API)
builder.Services.AddMassTransit(config =>
{
    config.UsingRabbitMq((ct, cfg) =>
    {
        cfg.Host(builder.Configuration["EventBusSettings:HostAddress"]);
    });
});
builder.Services.AddMassTransitHostedService();
```

Producer (Basket Service)

```csharp
// Basket.API/Controllers/BasketController.cs
public async Task<IActionResult> Checkout([FromBody] BasketCheckout
basketCheckout)
{
    // Get basket
    var query = new GetBasketByUserNameQuery(basketCheckout.UserName);
    var basket = await _mediator.Send(query);

    if (basket == null)
    {
        return BadRequest();
    }

    // Map and publish event
    var eventMsg = _mapper.Map<BasketCheckoutEvent>(basketCheckout);
    eventMsg.TotalPrice = basket.TotalPrice;
    await _publishEndpoint.Publish(eventMsg);

    // Remove basket
    var deletecmd = new DeleteBasketByUserNameCommand(basketCheckout.UserName);
    await _mediator.Send(deletecmd);

    return Accepted();
}
```

Consumer (Ordering Service)

```csharp
// Ordering.API/EventBusConsumer/BasketOrderingConsumer.cs
public class BasketOrderingConsumer : IConsumer<BasketCheckoutEvent>
{
    public async Task Consume(ConsumeContext<BasketCheckoutEvent> context)
    {
        using var scope = _logger.BeginScope("Consume Basket Checkout Event For
{correlationId}", context.Message.CorrelationId);
        var cmd = _mapper.Map<CheckoutOrderCommand>(context.Message);
        var result = await _mediator.Send(cmd);
        _logger.LogInformation("Basket Checkout Event completed !!");
    }
}
```

Event Versioning (V2)
```csharp
// Ordering.API/EventBusConsumer/BasketOrderingConsumerV2.cs
public class BasketOrderingConsumerV2 : IConsumer<BasketCheckoutEventV2>
{
    public async Task Consume(ConsumeContext<BasketCheckoutEventV2> context)
    {
        using var scope = _logger.BeginScope("Consuming basket Checkout Event for
{correlationId} with version 2");
        var cmd = _mapper.Map<CheckoutOrderCommandV2>(context.Message);
        var result = await _mediator.Send(cmd);
        _logger.LogInformation("Basket checkout Event completed with version
2!!!!");
    }
}
```

MassTransit Configuration (Ordering API)
```csharp
// Program.cs (Ordering.API)
builder.Services.AddMassTransit(config =>
{
    config.AddConsumer<BasketOrderingConsumer>();
    config.AddConsumer<BasketOrderingConsumerV2>();

    config.UsingRabbitMq((ctx, cfg) =>
    {
        cfg.Host(builder.Configuration["EventBusSettings:HostAddress"]);

        // V1 Queue
        cfg.ReceiveEndpoint(EventBusConstants.BasketCheckoutQueue, c =>
        {
            c.ConfigureConsumer<BasketOrderingConsumer>(ctx);
        });

        // V2 Queue
        cfg.ReceiveEndpoint(EventBusConstants.BasketCheckoutQueueV2, c =>
        {
            c.ConfigureConsumer<BasketOrderingConsumerV2>(ctx);
        });
    });
});
```

```

#### Docker Configuration
```yaml
# docker-compose.override.yml
rabbitmq:
  container_name: rabbitmq
  restart: always
  ports:
    - "5672:5672"    # AMQP port
    - "15672:15672"  # Management UI
```

RabbitMQ Management UI
```
http://localhost:15672
Username: guest
Password: guest
```

Benefits in Your Project
- **Decoupling** - Basket and Ordering services are loosely coupled
- **Asynchronous processing** - Basket checkout returns immediately
- **Reliability** - Messages persisted in RabbitMQ
- **Scalability** - Multiple Ordering instances can consume
- **Event versioning** - V1 and V2 queues for backward compatibility

---

Saga Pattern

Theory
Saga Pattern is a design pattern for managing distributed transactions across multiple microservices. It breaks a transaction into a sequence of local transactions, each updating data within a single service. If a step fails, compensating transactions undo the changes made by preceding transactions.

Problem Solved
In microservices, ACID transactions don't work across service boundaries because:
- Services have separate databases
- No distributed transaction coordinator
- Network failures can occur
- Services can be unavailable
```

Saga Pattern provides **eventual consistency** instead of immediate consistency.

Saga Types

1. Choreography-Based Saga
Services communicate via events without a central coordinator.

```
Service A → Event → Service B → Event → Service C
          ↓         ↓         ↓
        Comp      Comp      Comp
```

Pros:
- Simple to implement
- No single point of failure
- Decentralized

Cons:
- Hard to understand flow
- Complex error handling
- No central orchestration

2. Orchestration-Based Saga
A central coordinator (orchestrator) manages the saga.

```
Orchestrator → Service A → Orchestrator → Service B → Orchestrator → Service C
     ↓              ↓              ↓              ↓
   Comp           Comp           Comp           Comp
```

Pros:
- Centralized logic
- Easier to understand flow
- Better error handling

Cons:
- Single point of failure
- More complex to implement

Implementation (Not in Your Project)

Choreography-Based Example

```csharp
// Order Service
public class OrderService
{
    public async Task CreateOrder(Order order)
    {
        // Create order
        await _orderRepository.CreateOrder(order);

        // Publish OrderCreated event
        await _eventBus.Publish(new OrderCreatedEvent { OrderId = order.Id });
    }
}

// Payment Service
public class PaymentConsumer : IConsumer<OrderCreatedEvent>
{
    public async Task Consume(ConsumeContext<OrderCreatedEvent> context)
    {
        try
        {
            await _paymentService.ProcessPayment(context.Message.OrderId);
            await _eventBus.Publish(new PaymentCompletedEvent { OrderId =
context.Message.OrderId });
        }
        catch
        {
            await _eventBus.Publish(new PaymentFailedEvent { OrderId =
context.Message.OrderId });
        }
    }
}

// Order Service (Compensation)
public class OrderCompensationConsumer : IConsumer<PaymentFailedEvent>
{
    public async Task Consume(ConsumeContext<PaymentFailedEvent> context)
    {
```

```
        await _orderService.UpdateOrderStatus(context.Message.OrderId,
OrderStatus.Cancelled);
    }
}
```

Benefits
- **Distributed transactions** - Across multiple microservices
- **Eventual consistency** - Acceptable for most business cases
- **Resilience** - Compensating transactions handle failures
- **Scalability** - Each service handles its own part

CQRS Pattern

Theory
CQRS (Command Query Responsibility Segregation) is a design pattern that separates read and write operations for a data store. It uses separate models for updating (commands) and reading (queries) information.

Core Concept
- **Commands** - Change data (Create, Update, Delete)
- **Queries** - Read data (Get, Search, List)

Benefits
- **Performance optimization** - Separate models for read/write
- **Separation of concerns** - Clear business logic separation
- **Flexibility** - Different databases for read/write
- **Testability** - Isolated handlers

Implementation in Your Project

Commands (Write Operations)
```csharp
// CreateShoppingCartCommand.cs
public class CreateShoppingCartCommand : IRequest<ShoppingCartResponse>
{
    public string UserName { get; set; }
    public List<ShoppingCartItem> Items { get; set; }
}
```

```
// DeleteBasketByUserNameCommand.cs
public class DeleteBasketByUserNameCommand : IRequest<Unit>
{
    public string UserName { get; set; }
}
```

Queries (Read Operations)
```csharp
// GetBasketByUserNameQuery.cs
public class GetBasketByUserNameQuery : IRequest<ShoppingCartResponse>
{
    public string UserName { get; set; }
}
```

Command Handlers (Write Logic)
```csharp
// CreateShoppingCartCommandHandler.cs
public class CreateShoppingCartCommandHandler :
IRequestHandler<CreateShoppingCartCommand, ShoppingCartResponse>
{
    public async Task<ShoppingCartResponse> Handle(CreateShoppingCartCommand
request, CancellationToken cancellationToken)
    {
        // Business logic: Apply discounts via gRPC
        foreach (var item in request.Items)
        {
            var coupon = await
_discountGrpcService.GetDiscount(item.ProductName);
            item.Price -= coupon.Amount;
        }

        // Write operation: Update basket in Redis
        var shoppingCart = await _basketRepository.UpdateBasket(new ShoppingCart
{
            UserName = request.UserName,
            Items = request.Items
        });

        return _mapper.Map<ShoppingCartResponse>(shoppingCart);
    }
}
```

```

#### Query Handlers (Read Logic)
```csharp
// GetBasketByUserNameHandler.cs
public class GetBasketByUserNameHandler :
IRequestHandler<GetBasketByUserNameQuery, ShoppingCartResponse>
{
    public async Task<ShoppingCartResponse> Handle(GetBasketByUserNameQuery
request, CancellationToken cancellationToken)
    {
        // Read operation: Get basket from Redis
        var shoppingCart = await _basketRepository.GetBasket(request.UserName);
        return _mapper.Map<ShoppingCartResponse>(shoppingCart);
    }
}
```

Controller Usage
```csharp
// BasketController.cs
[HttpGet]
public async Task<ActionResult<ShoppingCartResponse>> GetBasket(string userName)
{
    var query = new GetBasketByUserNameQuery(userName);
    var basket = await _mediator.Send(query);
    return Ok(basket);
}

[HttpPost("CreateBasket")]
public async Task<ActionResult<ShoppingCartResponse>> UpdateBasket([FromBody]
CreateShoppingCartCommand createShoppingCartCommand)
{
    var basket = await _mediator.Send(createShoppingCartCommand);
    return Ok(basket);
}
```

CQRS with Validation (Ordering Service)
```csharp
// ValidationBehaviour.cs
public class ValidationBehaviour<TRequest, TResponse> :
IPipelineBehavior<TRequest, TResponse>
```
```

```
    where TRequest : IRequest<TResponse>
{
    public async Task<TResponse> Handle(TRequest request,
RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
    {
        if (_validators.Any())
        {
            var context = new ValidationContext<TRequest>(request);
            var validationResults = await Task.WhenAll(_validators.Select(v =>
v.ValidateAsync(context, cancellationToken)));
            var failure = validationResults.SelectMany(e => e.Errors).Where(f =>
f != null).ToList();

            if (failure.Count != 0)
            {
                throw new FluentValidation.ValidationException(failure);
            }
        }
        return await next();
    }
}
```

Benefits in Your Project
- **Clear separation** - Commands for write, queries for read
- **Validation pipeline** - FluentValidation with MediatR behaviors
- **Type safety** - Compile-time checking
- **Testability** - Isolated handlers

Mediator Pattern

Theory
Mediator Pattern is a behavioral design pattern that defines an object (mediator) that encapsulates how a set of objects interact. It promotes loose coupling by preventing objects from referring to each other explicitly.

Core Concept
Instead of objects communicating directly, they communicate through a mediator.

Without Mediator (Direct Coupling)
```
```

```
Object A → Object B → Object C → Object D
         ↓         ↓         ↓
      Tight coupling, hard to maintain
```

With Mediator (Loose Coupling)
```
Object A → Mediator ← Object B
Object C → Mediator ← Object D
         ↓
    Centralized coordination
```

MediatR in .NET

Request Types

IRequest (Single Response)
```csharp
public class GetUserQuery : IRequest<User>
{
    public int Id { get; set; }
}
```

IRequest<Unit> (No Response)
```csharp
public class DeleteUserCommand : IRequest<Unit>
{
    public int Id { get; set; }
}
```

INotification (Multiple Handlers)
```csharp
public class UserCreatedEvent : INotification
{
    public int UserId { get; set; }
}
```

Implementation in Your Project

MediatR Configuration
```csharp
// Program.cs (Basket.API)
builder.Services.AddMediatR(cfg =>
cfg.RegisterServicesFromAssemblies(assemblies));

var assemblies = new Assembly[]
{
    Assembly.GetExecutingAssembly(),
    typeof(CreateShoppingCartCommandHandler).Assembly
};
```

Pipeline Behaviors (Ordering Service)
```csharp
// ServiceRegistration.cs
services.AddMediatR(cfg =>
cfg.RegisterServicesFromAssembly(Assembly.GetExecutingAssembly()));
services.AddValidatorsFromAssembly(Assembly.GetExecutingAssembly());
services.AddTransient(typeof(IPipelineBehavior<,>),
typeof(ValidationBehaviour<,>));
services.AddTransient(typeof(IPipelineBehavior<,>),
typeof(UnhandledExceptionBehaviour<,>));
```

Validation Behavior
```csharp
// ValidationBehaviour.cs
public class ValidationBehaviour<TRequest, TResponse> :
IPipelineBehavior<TRequest, TResponse>
    where TRequest : IRequest<TResponse>
{
    public async Task<TResponse> Handle(TRequest request,
RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
    {
        if (_validators.Any())
        {
            var context = new ValidationContext<TRequest>(request);
            var validationResults = await Task.WhenAll(_validators.Select(v =>
v.ValidateAsync(context, cancellationToken)));
            var failure = validationResults.SelectMany(e => e.Errors).Where(f =>
f != null).ToList();
```

```
            if (failure.Count != 0)
            {
                throw new FluentValidation.ValidationException(failure);
            }
        }
        return await next();
    }
}
```

Exception Behavior
```csharp
// UnhandledExceptionBehaviour.cs
public class UnhandledExceptionBehaviour<TRequest, TResponse> :
IPipelineBehavior<TRequest, TResponse>
    where TRequest : IRequest<TResponse>
{
    public async Task<TResponse> Handle(TRequest request,
RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
    {
        try
        {
            return await next();
        }
        catch (Exception ex)
        {
            var requestName = typeof(TRequest).Name;
            _logger.LogError(ex, $"Unhandled exception occurred with Request
Name: {requestName}, {request}");
            throw;
        }
    }
}
```

Benefits in Your Project
- **Loose coupling** - Controllers don't depend on handlers
- **Cross-cutting concerns** - Validation, exception handling, logging
- **Testability** - Easy to test in isolation
- **Maintainability** - Clear separation of concerns

API Versioning

Theory
API Versioning is the practice of managing changes to an API over time by providing different versions of the API to maintain backward compatibility while introducing new features or breaking changes.

Versioning Strategies

1. URL Path Versioning
```
/api/v1/products
/api/v2/products
```

2. Query String Versioning
```
/api/products?version=1
/api/products?version=2
```

3. Header Versioning
```
GET /api/products
Headers:
  api-version: 1
```

4. Content Negotiation
```
GET /api/products
Headers:
  Accept: application/vnd.myapi.v1+json
```

Implementation in Your Project

Configuration
```csharp
// Program.cs (Basket.API)
builder.Services
    .AddApiVersioning(options =>
    {
```

```
        options.ReportApiVersions = true;
        options.AssumeDefaultVersionWhenUnspecified = true;
        options.DefaultApiVersion = new ApiVersion(1, 0);
    })
    .AddApiExplorer(options =>
    {
        options.GroupNameFormat = "'v'VVV";
        options.SubstituteApiVersionInUrl = true;
    });
```

Swagger Configuration
```csharp
// Program.cs
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new Microsoft.OpenApi.Models.OpenApiInfo { Title =
"Basket.API", Version = "v1" });
    c.SwaggerDoc("v2", new Microsoft.OpenApi.Models.OpenApiInfo { Title =
"Basket.API", Version = "v2" });
});

app.UseSwaggerUI(c =>
{
    c.SwaggerEndpoint("/swagger/v1/swagger.json", "Basket.API v1");
    c.SwaggerEndpoint("/swagger/v2/swagger.json", "Basket.API v2");
});
```

V1 Controller
```csharp
// BasketController.cs
[ApiVersion("1")]
public class BasketController : ApiController
{
    [HttpPost("CreateBasket")]
    public async Task<ActionResult<ShoppingCartResponse>> UpdateBasket([FromBody]
CreateShoppingCartCommand createShoppingCartCommand)
    {
        var basket = await _mediator.Send(createShoppingCartCommand);
        return Ok(basket);
    }
}
```

```

#### V2 Controller
```csharp
// Controllers/V2/BasketController.cs
[ApiVersion("2")]
[Route("api/v{version:apiVersion}/[controller]")]
public class BasketController : ControllerBase
{
    [HttpPost("Checkout")]
    public async Task<IActionResult> Checkout([FromBody] BasketCheckoutV2
basketCheckout)
    {
        // V2 implementation with simplified event
        var eventMsg = _mapper.Map<BasketCheckoutEventV2>(basketCheckout);
        await _publishEndpoint.Publish(eventMsg);
        return Accepted();
    }
}
```

Version Differences
```csharp
// V1 Event (Full)
public class BasketCheckoutEvent : BaseIntegrationEvent
{
    public string? UserName { get; set; }
    public decimal? TotalPrice { get; set; }
    public string? FirstName { get; set; }
    public string? LastName { get; set; }
    public string? EmailAddress { get; set; }
    public string? AddressLine { get; set; }
    public string? Country { get; set; }
    public string? State { get; set; }
    public string? ZipCode { get; set; }
    public string? CardName { get; set; }
    public string? CardNumber { get; set; }
    public string? Expiration { get; set; }
    public string? Cvv { get; set; }
    public int? PaymentMethod { get; set; }
}

// V2 Event (Simplified)
```
```

```
public class BasketCheckoutEventV2 : BaseIntegrationEvent
{
    public string? UserName { get; set; }
    public decimal? TotalPrice { get; set; }
}
```

Benefits in Your Project
- **Backward compatibility** - V1 clients continue working
- **Gradual migration** - New clients can use V2
- **Clear versioning** - Version visible in URL
- **Swagger support** - Separate docs for each version

Roles/Authorization

Theory
Authorization is the process of determining what permissions an authenticated user has. It answers the question: "What is this user allowed to do?"

Authorization Approaches

1. Role-Based Authorization (RBAC)
Users are assigned roles, and roles have permissions.

```
User → Role → Permissions
```

2. Claim-Based Authorization
Users have claims (key-value pairs) that represent attributes.

```
User → Claims → Authorization
```

3. Policy-Based Authorization
Policies combine requirements (roles, claims, custom logic).

```
Policy = Requirements (Role + Claim + Custom Logic)
```

```

### ASP.NET Core Authorization

#### Role-Based
```csharp
[Authorize(Roles = "Admin")]
public class ProductController : ControllerBase
{
    // Only admins can access
}

[Authorize(Roles = "Admin,Manager")]
public IActionResult CreateProduct()
{
    // User must have both roles
}
```

Claim-Based
```csharp
[Authorize(Policy = "RequireEmailClaim")]
public class ProductController : ControllerBase
{
}

// Configure policy
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireEmailClaim", policy =>
        policy.RequireClaim("email"));
});
```

Policy-Based
```csharp
[Authorize(Policy = "CanCreateProduct")]
public IActionResult CreateProduct()
{
    // Complex policy requirements
}
```
```

Implementation (Not in Your Project)

JWT Authentication Setup

```csharp
// Program.cs
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = "yourdomain.com",
            ValidAudience = "yourdomain.com",
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes("your-secret-key"))
        };
    });

builder.Services.AddAuthorization();
app.UseAuthentication();
app.UseAuthorization();
```

Suggested Roles for Your Project

- **Admin** - Full access to all endpoints
- **Manager** - Can manage products and orders
- **User** - Can view products, create basket, checkout
- **Guest** - Can view products only

Policy Configuration

```csharp
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy => policy.RequireRole("Admin"));
    options.AddPolicy("ManagerOrAdmin", policy => policy.RequireRole("Admin",
"Manager"));
    options.AddPolicy("UserOrAbove", policy => policy.RequireRole("Admin",
"Manager", "User"));
});
```

Benefits
- **Security** - Protect sensitive endpoints
- **Access Control** - Granular permissions
- **Audit** - Track who did what
- **Compliance** - Meet regulatory requirements

MSAL (Microsoft Authentication Library)

Theory
MSAL is a set of libraries provided by Microsoft for authenticating users and acquiring tokens from Microsoft identity platforms (Azure AD, Azure AD B2C, Microsoft Accounts).

Supported Platforms
- **MSAL.NET** - .NET applications
- **MSAL.js** - JavaScript/TypeScript (Angular, React, Vue)
- **MSAL for Android** - Android applications
- **MSAL for iOS/macOS** - iOS/macOS applications

Microsoft Identity Platforms

1. Azure Active Directory (Azure AD)
- **Enterprise identity** - For organizations
- **Single Sign-On (SSO)** - Across Microsoft services
- **Multi-factor authentication (MFA)** - Enhanced security
- **Conditional Access** - Policies based on risk

2. Azure AD B2C
- **Consumer identity** - For customer-facing apps
- **Custom branding** - Your own login experience
- **Social providers** - Google, Facebook, Twitter, etc.

3. Microsoft Accounts
- **Personal identity** - Outlook, Xbox, OneDrive users
- **Consumer-focused** - Personal Microsoft accounts

Token Types

Access Token

```
- **Purpose:** Access protected resources (APIs)
- **Lifetime:** Short-lived (5-60 minutes)
- **Usage:** Bearer token in Authorization header

#### ID Token
- **Purpose:** Identity information about user
- **Lifetime:** Short-lived (5-60 minutes)
- **Usage:** Verify user identity

#### Refresh Token
- **Purpose:** Obtain new access tokens
- **Lifetime:** Long-lived (days to months)
- **Usage:** Silent token renewal

### Authentication Flows

#### Authorization Code Flow (Recommended)
```
User → Login Page → Azure AD → Authorization Code
       ↓
App → Exchange Code for Tokens → Azure AD → Access Token + Refresh Token
```

#### Client Credentials Flow (Service-to-Service)
```
App → Client ID + Secret → Azure AD → Access Token
```

### Implementation (Not in Your Project)

#### .NET Configuration
```csharp
// Program.cs
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApi(builder.Configuration.GetSection("AzureAd"));

builder.Services.AddAuthorization();
```

Configuration (appsettings.json)
```json
{
```
```

```
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "your-tenant-id",
    "ClientId": "your-client-id",
    "ClientSecret": "your-client-secret",
    "Domain": "yourdomain.onmicrosoft.com"
  }
}
```

Token Acquisition
```csharp
public class TokenService
{
    public async Task<string> GetAccessTokenAsync()
    {
        var app = ConfidentialClientApplicationBuilder.Create(clientId)
            .WithClientSecret(clientSecret)
            .WithAuthority(new Uri($"{instance}{tenantId}"))
            .Build();

        var result = await app.AcquireTokenForClient(scopes).ExecuteAsync();
        return result.AccessToken;
    }
}
```

Benefits
- **Enterprise-grade security** - Microsoft-managed
- **Single Sign-On** - One login for multiple apps
- **Multi-factor authentication** - Enhanced security
- **Conditional access** - Risk-based policies
- **Compliance** - GDPR, SOC 2 compliant

Seed Data

Theory
Seed Data is initial data that is loaded into a database when the application starts or during deployment. It provides a baseline dataset for development, testing, or production.

Seed Data Types

1. Reference Data (Static)
Data that rarely changes:
- Countries, states, cities
- Product categories
- Payment methods
- Order statuses

2. Master Data (Business)
Core business data:
- Products
- Discounts
- Brands
- Product types

3. Configuration Data
System settings:
- Feature flags
- App settings
- User roles

Implementation in Your Project

Catalog Service (MongoDB)
```csharp
// CatalogContextSeed.cs
public static class CatalogContextSeed
{
    public static void SeedData(IMongoCollection<Product> productCollection)
    {
        bool checkProducts = productCollection.Find(b => true).Any();
        string path = Path.Combine("Data", "SeedData", "products.json");

        if (!checkProducts)
        {
            var productsData = File.ReadAllText(path);
            var products =
JsonSerializer.Deserialize<List<Product>>(productsData);

            if (products != null)
            {
                foreach (var item in products)
```

```
                {
                    productCollection.InsertOneAsync(item);
                }
            }
        }
    }
}

// BrandContextSeed.cs and TypeContextSeed.cs follow same pattern
```

Ordering Service (SQL Server)
```csharp
// OrderContextSeed.cs
public static class OrderContextSeed
{
    public static async Task SeedAsync(OrderContext orderContext,
ILogger<OrderContextSeed> logger)
    {
        if (!orderContext.Orders.Any())
        {
            orderContext.Orders.AddRange(GetOrders());
            await orderContext.SaveChangesAsync();
            logger.LogInformation($"Ordering Database:
{typeof(OrderContext).Name} seeded!!!");
        }
    }

    private static IEnumerable<Order> GetOrders()
    {
        return new List<Order>
        {
            new()
            {
                UserName = "rahul",
                FirstName = "Rahul",
                LastName = "Sahay",
                EmailAddress = "rahulsahay@eCommerce.net",
                AddressLine = "Bangalore",
                Country = "India",
                TotalPrice = 750,
                State = "KA",
                ZipCode = "560001",
```

```
                CardName = "Visa",
                CardNumber = "1234567890123456",
                CreatedBy = "Rahul",
                Expiration = "12/25",
                Cvv = "123",
                PaymentMethod = 1,
                LastModifiedBy = "Rahul",
                LastModifiedDate = new DateTime(),
            }
        };
    }
}
```

Database Migration Extension
```csharp
// InfraServices.cs
public static class InfraServices
{
    public static IApplicationBuilder MigrateDatabase<TContext>(this
IApplicationBuilder builder, Action<TContext, IServiceProvider> seeder) where
TContext : DbContext
    {
        using var scope = builder.ApplicationServices.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<TContext>();
        var services = scope.ServiceProvider;

        context.Database.Migrate();  // Apply EF migrations
        seeder(context, services);   // Seed data

        return builder;
    }
}
```

Usage in Program.cs
```csharp
// Ordering.API/Program.cs
app.MigrateDatabase<OrderContext>((context, services) =>
{
    var logger = services.GetService<ILogger<OrderContextSeed>>();
    OrderContextSeed.SeedAsync(context, logger).Wait();
});
```

```

#### JSON Seed Data Files
```json
// products.json
[
  {
    "id": "602d2149e773f2a3990b47f5",
    "name": "Adidas Quick Force Indoor Badminton Shoes",
    "summary": "Badminton shoes for indoor courts",
    "description": "High-performance badminton shoes with excellent grip and cushioning",
    "imageFile": "images/products/adidas_shoe-1.png",
    "price": 3500,
    "brands": {
      "id": "602d2149e773f2a3990b47f1",
      "name": "Adidas"
    },
    "types": {
      "id": "602d2149e773f2a3990b47f3",
      "name": "Shoes"
    }
  }
]
```

Benefits in Your Project
- **Consistent environment** - Same data across all environments
- **Faster setup** - No manual data entry
- **Version controlled** - Seed data in code/JSON files
- **Automated** - Runs on application startup

Factory Pattern

Theory
Factory Pattern is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.

Simple Factory
```csharp
```
```

```csharp
public class NotificationFactory
{
    public INotification CreateNotification(string type)
    {
        return type switch
        {
            "email" => new EmailNotification(),
            "sms" => new SmsNotification(),
            "push" => new PushNotification(),
            _ => throw new ArgumentException($"Unknown notification type:
{type}")
        };
    }
}
```

Factory Method
```csharp
public abstract class NotificationCreator
{
    public abstract INotification CreateNotification();

    public void SendNotification(string message)
    {
        var notification = CreateNotification();
        notification.Send(message);
    }
}

public class EmailNotificationCreator : NotificationCreator
{
    public override INotification CreateNotification()
    {
        return new EmailNotification();
    }
}
```

Use Cases in Microservices

Database Connection Factory
```csharp
public interface IDbConnectionFactory
```

```
{
    IDbConnection CreateConnection();
}

public class SqlConnectionFactory : IDbConnectionFactory
{
    private readonly string _connectionString;

    public SqlConnectionFactory(string connectionString)
    {
        _connectionString = connectionString;
    }

    public IDbConnection CreateConnection()
    {
        return new SqlConnection(_connectionString);
    }
}
```

Service Client Factory
```csharp
public class GrpcClientFactory : IServiceClientFactory
{
    public IServiceClient CreateClient(string serviceType)
    {
        return serviceType switch
        {
            "discount" => CreateDiscountClient(),
            "inventory" => CreateInventoryClient(),
            _ => throw new ArgumentException($"Unknown service type:
{serviceType}")
        };
    }
}
```

Benefits
- **Loose coupling** - Client doesn't know concrete classes
- **Single responsibility** - Factory handles creation
- **Open-Closed Principle** - Extensible without modification
- **Testability** - Easy to mock factories

```
---

## Abstract Factory Pattern

### Theory
**Abstract Factory Pattern** is a creational design pattern that lets you produce families of related objects without specifying their concrete classes.

### Factory vs Abstract Factory

| Aspect | Factory Pattern | Abstract Factory Pattern |
|--------|-----------------|--------------------------|
| **Creates** | Single object | Family of related objects |
| **Complexity** | Simple | Complex |
| **Use Case** | One type of product | Multiple related product types |

### Abstract Factory Example

#### UI Component Factory
```csharp
// Abstract Products
public interface IButton
{
    void Render();
    void Click();
}

public interface ITextBox
{
    void Render();
    void SetText(string text);
}

// Abstract Factory
public interface IUIFactory
{
    IButton CreateButton();
    ITextBox CreateTextBox();
}

// Concrete Factories
public class WindowsUIFactory : IUIFactory
```
```

```
{
    public IButton CreateButton()
    {
        return new WindowsButton();
    }

    public ITextBox CreateTextBox()
    {
        return new WindowsTextBox();
    }
}

public class MacUIFactory : IUIFactory
{
    public IButton CreateButton()
    {
        return new MacButton();
    }

    public ITextBox CreateTextBox()
    {
        return new MacTextBox();
    }
}
```

Use Cases in Microservices

Database Family Factory
```csharp
// Abstract Products
public interface IDbConnection { void Connect(); void Disconnect(); }
public interface IDbCommand { void SetCommand(string command); void Execute(); }
public interface IDbTransaction { void Begin(); void Commit(); void Rollback(); }

// Abstract Factory
public interface IDbFactory
{
    IDbConnection CreateConnection();
    IDbCommand CreateCommand();
    IDbTransaction CreateTransaction();
}
```

```csharp
// Concrete Factories
public class SqlDbFactory : IDbFactory
{
    public IDbConnection CreateConnection() => new SqlConnection();
    public IDbCommand CreateCommand() => new SqlCommand();
    public IDbTransaction CreateTransaction() => new SqlTransaction();
}

public class PostgreSqlDbFactory : IDbFactory
{
    public IDbConnection CreateConnection() => new PostgreSqlConnection();
    public IDbCommand CreateCommand() => new PostgreSQLCommand();
    public IDbTransaction CreateTransaction() => new PostgreSQLTransaction();
}
```

Message Broker Family Factory

```csharp
// Abstract Products
public interface IMessageProducer { void Publish(string topic, object message); }
public interface IMessageConsumer { void Subscribe(string topic, Action<object>
handler); }
public interface IMessageQueue { void CreateQueue(string queueName); }

// Abstract Factory
public interface IMessageBrokerFactory
{
    IMessageProducer CreateProducer();
    IMessageConsumer CreateConsumer();
    IMessageQueue CreateQueue();
}

// Concrete Factories
public class RabbitMqFactory : IMessageBrokerFactory
{
    public IMessageProducer CreateProducer() => new RabbitMqProducer();
    public IMessageConsumer CreateConsumer() => new RabbitMqConsumer();
    public IMessageQueue CreateQueue() => new RabbitMqQueue();
}

public class KafkaFactory : IMessageBrokerFactory
{
    public IMessageProducer CreateProducer() => new KafkaProducer();
```

```
    public IMessageConsumer CreateConsumer() => new KafkaConsumer();
    public IMessageQueue CreateQueue() => new KafkaQueue();
}
```

Benefits
- **Family creation** - Creates related objects together
- **Loose coupling** - Client doesn't know concrete classes
- **Open-Closed Principle** - Extensible without modification
- **Consistency** - Products from same factory are compatible

---

Summary

Topics Implemented in Your Project
1. ☑ **Redis** - Basket storage with IDistributedCache
2. ☑ **gRPC** - Basket → Discount communication
3. ☑ **REST vs gRPC** - Hybrid approach (REST for frontend, gRPC for inter-service)
4. ☑ **Elasticsearch** - Logging with Serilog + Kibana
5. ☑ **RabbitMQ** - Event-driven communication with MassTransit
6. ☑ **CQRS** - Implemented with MediatR in all services
7. ☑ **Mediator** - MediatR for command/query dispatching
8. ☑ **API Versioning** - URL path versioning with v1 and v2
9. ☑ **Seed Data** - Catalog (MongoDB JSON), Ordering (SQL Server code)

Topics Not Implemented (Theory Only)
10. ☑ **Hangfire** - Background services
11. ☑ **Polly** - Resilience patterns (installed but not configured)
12. ☑ **Saga** - Distributed transaction pattern
13. ☑ **Roles/Authorization** - Not implemented (no auth)
14. ☑ **MSAL** - Microsoft Authentication
15. ☑ **Factory Pattern** - Design pattern
16. ☑ **Abstract Factory** - Design pattern

Key Architectural Patterns in Your Project

Clean Architecture:
- API Layer (Controllers)
- Application Layer (CQRS with MediatR)

```
- Core Layer (Entities, Repositories)
- Infrastructure Layer (Data access, external services)

**CQRS + MediatR:**
- Commands (write operations)
- Queries (read operations)
- Handlers (business logic)
- Pipeline behaviors (validation, exception handling)

**Event-Driven Architecture:**
- RabbitMQ message broker
- MassTransit abstraction
- Event versioning (V1, V2)
- Decoupled services

**Polyglot Persistence:**
- Redis (Basket)
- MongoDB (Catalog)
- PostgreSQL (Discount)
- SQL Server (Ordering)

**Observability:**
- Elasticsearch logging
- Kibana visualization
- Serilog structured logging

### Recommendations for Improvement

**High Priority:**
1. **Add Authentication/Authorization** - MSAL + Role-based access
2. **Implement Polly** - Add resilience to gRPC/HTTP calls
3. **Add Health Checks** - All services should have health checks

**Medium Priority:**
4. **API Gateway** - Single entry point for frontend
5. **Distributed Tracing** - OpenTelemetry/Jaeger
6. **Add Hangfire** - Background job processing

**Low Priority:**
7. **Implement Saga** - Full distributed transaction pattern
8. **Add Factory Pattern** - For service client creation
9. **Add Abstract Factory** - For database/message broker families
```

```
---

This comprehensive guide covers all 16 microservices topics with theory and
practical examples from your project. Each topic includes implementation details,
code examples, and benefits specific to your microservices architecture.
```